

RECEPTA PDDL (Pregunta 2 del final — 5 punts)

Estructura sempre igual: a) Domini (~4p) + b) Problema (~1p).
El domini és genèric (s'ha de poder estendre); el problema és una "foto" concreta.

0. Els 5 principis d'or (els repeteixen a TOTES les solucions)

Posa una frase d'introducció a l'apartat a) dient que segueixes aquests criteris. **Sumen punts sols.**

1. **Minimitzar el nombre d'operadors** i el seu factor de ramificació.
2. **Evitar operadors amb paràmetres similars** → que l'existència d'objectes dirigeixi la instanciació.
3. **Usar tipus** a les variables per reduir objectes a comprovar.
4. **Evitar exists i forall** a precondicions/efectes (augmenten el factor de ramificació amb "variables ocultes"). → Es resolen amb **predicats-filtre**.
5. **Prioritzar execució eficient encara que la representació sigui més complexa** (afegeix predicats/tipus si ajuden el planificador).

1. Estructura del fitxer DOMINI

```
(define (domain NOM)
  (:requirements :adl :typing)

  (:types ...jerarquia de tipus... )

  (:predicates ...)

  (:action ...
    :parameters (...)
    :precondition (and ...)
    :effect (and ...))
)
```

Pas 1 — TIPUS (:types)

- Identifica les "categories" d'objectes de l'enunciat (vehicles, llocs, paquets, comensals...).
- Fes **subtipus** quan un grup té un comportament/restricció diferent.
- Ex. material sanitari: `pedidoNormal` i `pedidoRefrigerado` són subtipus de `pedido`; `vehiculoNormal`/`vehiculoRefrigerado` subtipus de `vehiculo`.
- **Truc fort:** els tipus poden substituir predicats-filtre. Si un recurs refrigerat només pot anar en vehicle refrigerat, fes-ho amb tipus (no amb `imply`).

Pas 2 — PREDICATS (:predicates)

Dos famílies:

- **De fet / estat del món:** on és cada cosa, què conté, relacions fixes.

- (estacionado ?ve - vehiculo ?d - direccion), (en ?p - pedido ?l - lugar), (dentro ?s - slot

?ve - vehiculo), (conectado ?l1 ?l2), (pertenece_a ?silla ?mesa)...

- **Predicats-FILTRE (clau per evitar forall/exists):** booleans que et diuen "ja fet / lliure / ocupat".
- (libre ?s - slot) → saber si un slot està buit sense recórrer tots els paquets.
- (servido ?p) / (colocado ?c) → per la condició objectiu i per filtrar a la precondició.
- (asignada ?silla) → silla ja ocupada.

Regla mnemotècnica: cada cop que necessitis preguntar "existeix algun X tal que..." o "per a tot X...", crea un predicat-filtre booleà i el mantens actualitzat als efectes.

Pas 3 — ACCIONS (:action)

Una acció per cada "verb" que l'enunciat demana modelar explícitament (moure's, recollir, deixar, entregar, fabricar, col·locar...).

Plantilla:

```
(:action recoger
:parameters (?pe - pedido ?ve - vehiculo ?s - slot ?d - direccion)
:precondition (and
  (not (servido ?pe))      ; filtre: encara no fet
  (en ?pe ?d)              ; el paquet és aquí
  (estacionado ?ve ?d)     ; el vehicle és aquí ← evita "teletransport"
  (dentro ?s ?ve) (libre ?s))
:effect (and
  (en ?pe ?s) (not (en ?pe ?d))
  (not (libre ?s))))
```

Errors típics que has d'evitar (i comentar que evites):

- [X] Que recollir/entregar **mogui** el vehicle → impedeix recollir 2 coses al mateix lloc. Exigeix (estacionado ?ve ?d) a la precondició, no ho moguis a l'efecte.
- [X] "Teletransport màgic": sempre comprova que vehicle i paquet siguin a la mateixa direcció.
- [X] Oblidar el predicat-filtre a l'efecte (deixar **libre/servido** desactualitzat).

2. Estructura del fitxer PROBLEMA

```
(define (problem NOM-concret)
  (:domain NOM)
  (:objects
    ...llista objectes per tipus... )
  (:init
    ...estat inicial: una foto literal de l'enunciat... )
  (:goal (and
    (forall (?p - pedido) (servido ?p))
    (forall (?v - vehiculo) (estacionado ?v parking)))))
```

- **:objects** — un objecte per cada cosa concreta, agrupats per tipus. Usa els subtipus fulla (p1 - pGuantes).
- **:init** — descriu literalment: on és cada cosa (en, estacionado), relacions (destino, pertenece_a, conectado, izquierda_de), slots de cada vehicle (dentro + libre).
- **:goal** — usa els predicats-filtre. **forall** **SÍ es pot fer servir al goal** (en Fast-Forward s'instancia un sol cop, cost lineal). Permet canviar el nombre d'objectes sense reescriure el goal.

3. Casos especials que han caigut

- **Capacitat de contenidors** (robots, vans): modela cada contenidor com un objecte `slot` amb `(dentro slot vehicle) + (libre slot)`. **No cal fluents**. Un robot amb 2 contenidors = 2 slots.
- **Mapa d'accessibilitat** (Minecraft, Golem, viatges): predicat `(conectado ?l1 ?l2)` i acció `ir_a` amb precondició `(conectado ?ori ?des)`.
- **Receptes de fabricació** (Minecraft, gremi màgic): acció `fabricar` amb precondicions que exigeixen tenir els ingredients `(tiene ?ingr)` i l'estació col·locada `(estacion_colocada ?est ?lugar)`. Recorda: en alguns enunciats fabricar **no consumeix** ingredients (simplificació).
- **Estacions de processat** (Minecraft): acció `colocar_estacion` que treu l'objecte de l'inventari i el deixa disponible al lloc.
- **Ordre/precedència** (bany ShareBath, banquet): predicats com `(hecho ?tarea ?persona)` i precondicions que exigeixen tasques prèvies (dutxar-se abans de pentinar-se).
- **Recursos compartits** (bany: dutxa, secador): accions `coger/dejar recurs` + predicat `(libre_recurso ?r)` per evitar conflictes.
- **Restriccions binàries entre objectes** (banquet: "necessita al costat" / "no suporta"): predicats `(necesita ?x ?y)`, `(nosoporta ?x ?y)` + paràmetres extra `?sizq ?sder` a l'operador per les cadenes veïnes.

4. Checklist abans d'entregar

- [] He posat la frase dels 5 principis d'or?
 - [] Tots els predicats-filtre s'actualitzen a CADA efecte que toca?
 - [] Cap acció mou el vehicle/agent quan recull/entrega?
 - [] El `:goal` usa predicats-filtre (no exists)?
 - [] El domini és **extensible** (més objectes sense tocar el domini)? Diques-ho explícitament.
 - [] He explicat el significat de cada predicat i de cada decisió? (ho valoren molt)
- ...